

Câu 1. Hiểu bản chất công cụ

a) Điểm khác biệt cốt lõi:

AI IDE (như Cursor, GitHub Copilot) nhúng vào môi trường soạn thảo, hiểu ngữ cảnh toàn bộ codebase, biết file nào đang mở, function nào liên quan, có thể gợi ý inline và refactor theo luồng làm việc của dev.

AI Terminal (như Claude Code, Aider) nhận lệnh tự nhiên và tự thực thi, chạy bash, cài package, sửa nhiều file cùng lúc mà không cần dev chỉ từng bước. Tự chủ hơn, ít cần giám sát từng thao tác.

b) Tình huống phù hợp:

- AI IDE: Đang viết một service mới, cần gợi ý autocomplete đúng kiểu dữ liệu, refactor inline, xem lỗi ngay tại chỗ. → Cursor phù hợp vì hiểu context của file đang mở.
- AI Terminal: Cần scaffold nhanh một REST API (tạo folder, file, cài dependencies, generate boilerplate). → Claude Code phù hợp vì thực thi nhiều bước liên tiếp mà không cần dev click từng chỗ.

c) Vì sao không nên tách biệt hoàn toàn:

Trên thực tế hai loại bổ sung cho nhau trong một workflow. Ví dụ: dùng AI Terminal để scaffold project, sau đó mở trong Cursor để tinh chỉnh logic và review từng dòng. Nếu xem chúng là hai thế giới riêng, dev dễ dùng sai tool cho sai việc, dùng IDE để làm thứ cần tự động hóa (chậm), hoặc dùng Terminal để viết logic phức tạp (thiếu kiểm soát).

Câu 2. Tốc độ và kiểm soát

a) Tốc độ không đồng nghĩa an toàn hay chất lượng tốt hơn:

AI tăng tốc ở khâu sinh ra output, nhưng không tăng tốc ở khâu hiểu yêu cầu nghiệp vụ hay phát hiện lỗi tinh. Code sinh ra trong 3 giây vẫn có thể sai logic, thiếu edge case, hoặc có lỗ hổng bảo mật. Tốc độ cao còn làm dev ít đọc kỹ hơn, phản ứng không kịp khi có vấn đề.

b) Rủi ro lớn hơn: con người tin quá nhanh vào output AI

AI sinh sai là điều có thể dự đoán và đã biết trước, tool nào cũng có giới hạn. Nguy hiểm hơn là con người mất đi sự hoài nghi cần thiết. Khi nhìn thấy code "trông có vẻ đúng", chạy không báo lỗi, dev có xu hướng accept ngay. Đây là dạng lỗi không nhìn

thấy được, không có exception, không có test fail, chỉ có logic sai âm thầm chạy trong production.

Ví dụ thực tế: AI sinh hàm tính phần trăm giảm giá nhưng áp dụng sai thứ tự phép tính. Test pass vì test cũng do AI viết với cùng assumption sai.

Câu 3. Phản biện nhận định

"Nếu đã dùng AI sinh code, chỉ cần dùng thêm một AI khác sinh test case là đủ để yên tâm."

Nhận định này sai, vì:

- Coverage illusion: AI sinh test thường cover các happy path và case hiển nhiên, tạo cảm giác coverage cao (ví dụ 80% line coverage) nhưng bỏ qua edge case thực tế, input rỗng, ký tự đặc biệt, race condition. Nhìn vào số % coverage mà tưởng an toàn là ảo giác.
- Confirmation bias: AI sinh test dựa trên cùng code nó vừa sinh ra, có xu hướng viết test khớp với behavior hiện tại thay vì verify behavior đúng. Hai AI cùng nhìn một hướng, không ai đặt câu hỏi ngược lại.
- Thiếu hacker mindset: Tester giỏi không hỏi "code này có chạy không?" mà hỏi "làm sao để phá nó?". AI sinh test thường không chủ động thử injection, bypass auth, hay stress test với dữ liệu bất thường. Không có ai đứng ở phía đối nghịch để tấn công hệ thống.
- Test đúng theo code, chưa chắc đúng theo nghiệp vụ: Giả sử AI sinh route /admin không yêu cầu xác thực, test có thể pass hoàn toàn (gọi được, trả về 200), nhưng về nghiệp vụ đây là lỗ hổng bảo mật nghiêm trọng. AI không có spec, không có context tổ chức, chỉ có con người mới biết "đúng về nghiệp vụ" nghĩa là gì.

Kết luận: Dùng AI sinh test chỉ là thêm một lớp tốc độ, không phải thêm một lớp bảo đảm. Cần ít nhất: human review spec, test độc lập với người viết code, và test case từ góc nhìn người dùng thực tế.

Câu 4. Phân tích rủi ro quy trình

- Rủi ro 1: AI tự cài package không kiểm tra → cài thư viện lỗi thời, có CVE, hoặc typosquatting (tên gần giống package phổ biến nhưng là malware)
→ Audit package.json sau mỗi lần AI cài: kiểm tra version, nguồn gốc, chạy npm audit hoặc pip-audit

- Rủi ro 2: Route đăng nhập do AI sinh thiếu bảo mật → không có rate limiting, không hash password đúng cách, thiếu CSRF protection
→ Code review bắt buộc với checklist bảo mật, không merge route auth mà chưa qua mắt người
- Rủi ro 3: AI đọc CV PDF để lọc ứng viên → bias ẩn trong model, loại ứng viên tốt vì format CV khác chuẩn, vi phạm nguyên tắc tuyển dụng công bằng
→ Dùng AI như công cụ hỗ trợ sơ loại, không để AI ra quyết định cuối; HR review lại danh sách bị loại
- Rủi ro 4: AI tự chạy test và báo "pass" không có giám sát → test có thể không cover đúng nghiệp vụ, hoặc chính test cũng sai (như đã phân tích câu 3) — dev tin vào kết quả mà không đọc detail
→ Không accept "pass" như một kết quả cuối cùng: xem log chi tiết, kiểm tra test case list, và có ít nhất một smoke test thủ công với scenario thực